

Melody Vault — Windows Setup Guide

Setting up and running the playlist/video downloader on a Windows PC, using VS Code.

Overview

Four ways to pull media into a local downloads folder: a public Spotify playlist, a public Gaana playlist, a single YouTube video, or a whole YouTube playlist. No Spotify or Gaana account is required for any of it. Three ways to use it once set up: plain terminal (`main.py`), a desktop window (`gui.py`), or a rich local web page (`app.py`).

Part 1: Installing VS Code and Dependencies

Step 1: Install VS Code and the Python extension

Download VS Code from code.visualstudio.com. Open it, go to Extensions (`Ctrl+Shift+X`), and install the official “Python” extension from Microsoft.

Step 2: Open the project folder

File → Open Folder, and select the project folder. This makes it the workspace root so relative paths like `downloads/` and `.env` resolve correctly.

Step 3: Create and activate a virtual environment

Open the integrated terminal (`Ctrl+``) and run:

```
python -m venv venv
.\venv\Scripts\Activate.ps1
```

Note: If PowerShell blocks the script with an execution-policy error, run this once first: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`

Step 4: Install project dependencies

With the venv active:

```
pip install -r requirements.txt
```

Note: This installs `spotifyscraper`, `gaanaclient`, `google-api-python-client`, `yt-dlp`, `python-dotenv`, and `flask`.

Step 5: Install ffmpeg

Needed to extract and merge audio/video streams:

```
winget install ffmpeg
```

Note: Restart the terminal afterward, then confirm with: `ffmpeg -version`

Step 6: Install Deno (recommended)

Deno is a lightweight JavaScript runtime yt-dlp uses to solve a signature challenge YouTube requires. Without it, downloads still work but are slower and print repeated warnings.

1. Download `deno-x86_64-pc-windows-msvc.zip` from github.com/denoland/deno/releases/latest
2. Unzip it, e.g. into `C:\deno`
3. Add `C:\deno` to your PATH (search "Environment Variables" in Windows, edit the Path variable under User variables)
4. Restart your terminal and confirm: `deno --version`

Step 7: Get a YouTube API key

This is the only credential the project needs — no Spotify or Gaana account or login is required.

1. Go to console.cloud.google.com
2. Create a project → enable YouTube Data API v3
3. Credentials → Create Credentials → API key
4. Choose "Public data" when asked what you're accessing (not "User data") — this only does anonymous searches

Step 8: Set up your .env file

Copy the template and fill in your key:

```
copy .env.example .env
```

Note: Open `.env` in VS Code and paste in `YOUTUBE_API_KEY`. Leave `APP_USERNAME/APP_PASSWORD/SECRET_KEY` blank unless setting up remote access (Part 3).

Part 2: Running It

Terminal: `python main.py` — prompts for a source (Spotify, Gaana, YouTube video, or YouTube playlist) and a link.

Desktop GUI: `python gui.py` — a light-blue-and-gold Tkinter window with the same four modes, a track/video list, and a progress bar.

Web UI: `python app.py` — opens `http://127.0.0.1:5000` automatically in your default browser. Same four modes as tabs, with a live activity console and animated progress bar.

Part 3: Private Remote Access (Optional)

The web UI (`app.py`) has a login screen built in, so it can safely be reached from your phone over a private network — never expose it to the public internet.

Step 1: Set real credentials

In `.env`, fill in real values:

```
APP_USERNAME=your_chosen_username
APP_PASSWORD=a_real_password
SECRET_KEY=
python -c "import secrets; print(secrets.token_hex(32))"
```

Note: Without APP_PASSWORD set, the server refuses to bind beyond 127.0.0.1 — this is a deliberate safety check.

Step 2: Install Tailscale

Install Tailscale (tailscale.com/download) on this PC and on your phone, signed into both with the same account. This creates a private network between only your own devices.

Step 3: Run and connect

Start the server, then find this machine's Tailscale address:

```
python app.py  
tailscale ip -4
```

Note: From your phone (connected to Tailscale), visit <http://:5000> and sign in.

Part 4: Troubleshooting

ModuleNotFoundError after adding a new file/feature

requirements.txt was updated but not reinstalled. Run `pip install -r requirements.txt` again.

“Quota exceeded” / rateLimitExceeded error

The YouTube Data API's free tier allows about 100 searches per day, shared across the whole day and every run — not per playlist. Both `main.py` and `app.py` catch this, log a clear message, and stop cleanly. It resets at midnight Pacific Time; already-downloaded tracks are skipped automatically on the next run.

Pasted a playlist link into the “single video” option

This is detected and shows a clear error asking for a single video link instead — use the “YouTube playlist” mode for a whole playlist.

VS Code: “terminal environment injection is disabled”

Safe to dismiss. The project loads `.env` itself via `python-dotenv`; it doesn't rely on VS Code's terminal injection.

Repeated “No supported JavaScript runtime” warnings

Install Deno (Part 1, Step 6) — this is exactly what it fixes.

Downloads stall or fail on a large file

Check disk space first — high-resolution long videos can be several GB, and the merge step briefly needs room for the separate video and audio parts plus the final file. Interrupted downloads resume automatically on rerun via `yt-dlp`'s partial-file handling.